EE 66 Signals, Dynamics, and Information
Fall 2026 Note 10

10.1 Discrete-Time Dynamical Systems

So far in this course, we have built up tools for working with signals and vectors: inner products and norms, complex exponentials, and least squares. In this note, we turn to systems that change over time. Populations grow and shrink, water flows between reservoirs, and commuters move between cities every day. If we know the state of such a system today, can we predict where it will be tomorrow, or a year from now? A **dynamical system** is a mathematical model of how a system changes over time, and it is exactly the tool we need to answer this question.

We can model time as continuous, with the system evolving smoothly, or as discrete, with the system observed at specific time steps. In this note, we focus on **discrete-time dynamical systems**, where we take snapshots of the system at times $n = 0, 1, 2, \ldots$ and use a model to describe how each snapshot leads to the next. To specify such a system, we need three components:

1. **State variables**: the quantities that completely describe the system at each time step.
2. **Initial condition**: the values of the state variables at time $n = 0$.
3. **State evolution equation**: an equation that gives the state variables at time $n+1$ in terms of their values at time n.

10.1.1 A Scalar Example: Population Growth

Let's start with the simplest possible case, where the state is a single number. Let $m[n]$ be the number of deer in a nature reserve in year n. If the population grows by 8% per year, then

$$m[n+1] = 1.08\, m[n]. \tag{1}$$

This is a discrete-time dynamical system: the state is $m[n]$, the state evolves through multiplication by 1.08, and the initial condition is $m[0]$. If we start with $m[0] = 1000$ deer, iterating gives $m[1] = 1080$, $m[2] \approx 1166$, $m[3] \approx 1260$, and in general

$$m[n] = (1.08)^n\, m[0].$$

The same argument works for any state evolution equation $m[n+1] = a\, m[n]$: its solution $m[n] = a^n\, m[0]$ tells us the state at any time n. What happens to the state in the long run, as $n \to \infty$? The answer depends only on the size of a:

UCB EE 66, Fall 2026, Note 10, All Rights Reserved. This may not be publicly shared without explicit permission.

- $|a| < 1$: $m[n] \to 0$.
- $|a| = 1$: $|m[n]|$ stays constant.
- $|a| > 1$: $|m[n]| \to \infty$.

10.1.2 From Scalars to Vectors

Now suppose our nature reserve has deer *and* wolves, and the two populations affect each other: wolves eat deer, and more deer means more food for the wolves. The number of deer alone no longer describes the system, so we need to track both populations at once. We collect them into a **state vector**:

$$\mathbf{q}[n] = \begin{bmatrix} q_1[n] \\ q_2[n] \end{bmatrix}$$

where $q_1[n]$ is the deer population and $q_2[n]$ is the wolf population at time n. Since the populations interact, the next value of each one depends on both current values, and the evolution rule takes the form of a matrix multiplication:

$$\mathbf{q}[n+1] = \mathbf{A}\mathbf{q}[n]$$

The rest of this note develops the tools we need to analyze vector-valued systems like this one. Notice that the model $\mathbf{q}[n+1] = \mathbf{A}\mathbf{q}[n]$ is **linear**, since the next state is a linear function of the current state, and **time-invariant**, since the same matrix $\mathbf{A}$ is used at every time step. We call systems with both properties **linear time-invariant (LTI)** systems. We will define them formally later in the course, and in this note we focus exclusively on them.

10.2 The State of a System

Now that we have seen a few examples of dynamical systems, let's make precise what we mean by the state of a system.

Definition 10.1 (State of a System): The **state** of a system at time n is the minimal information needed at time n so that, together with any future inputs, we can determine the future behavior of the system.

In other words, the state is everything we need to know about the system right now in order to predict its future. In the deer-and-wolves example, the two populations are exactly this information: if we know both of them this year, the model tells us both of them next year. In general, we represent the state as a vector $\mathbf{q}[n] \in \mathbb{R}^N$, called the **state vector**, or the **state** for short. Each component $q_k[n]$ is called a **state variable**, and the number N of state variables is the **dimension** of the state space.

10.3 State Evolution Without Inputs

Now that we know what the state of a system is, we can describe how it changes over time. We begin with systems that evolve on their own, without any outside influence. The simplest model describes how the state evolves from one time step to the next via matrix multiplication:

$$\mathbf{q}[n+1] = \mathbf{A}\mathbf{q}[n], \quad n = 0, 1, 2, \ldots \tag{2}$$

where $\mathbf{A} \in \mathbb{R}^{N \times N}$ is the **state transition matrix** and $\mathbf{q}[0]$ is the **initial state**. The matrix $\mathbf{A}$ describes how the system evolves from one point in time to the next.

Starting from the initial state $\mathbf{q}[0]$, we can compute future states step by step:

$$\begin{aligned}
\mathbf{q}[1] &= \mathbf{A}\mathbf{q}[0] \\
\mathbf{q}[2] &= \mathbf{A}\mathbf{q}[1] = \mathbf{A}^2\mathbf{q}[0] \\
\mathbf{q}[3] &= \mathbf{A}\mathbf{q}[2] = \mathbf{A}^3\mathbf{q}[0]
\end{aligned}$$

Each step multiplies the state by one more factor of $\mathbf{A}$, so the pattern gives us a closed-form expression, just like $m[n] = a^n m[0]$ in the scalar case:

$$\boxed{\mathbf{q}[n] = \mathbf{A}^n \mathbf{q}[0]} \tag{3}$$

In other words, to find the state at any time n, all we need is the initial state and the n-th power of $\mathbf{A}$.

Example 10.1 (Two-City Commuter System): Consider two cities, Alphaville and Betatown, with commuters traveling between them each day. Let $q_1[n]$ be the number of commuters in Alphaville at time n and $q_2[n]$ the number in Betatown, so $\mathbf{q}[n] = \begin{bmatrix} q_1[n] \\ q_2[n] \end{bmatrix}$. Suppose that each day:

- 80% of commuters in Alphaville stay, and 20% move to Betatown.
- 50% of commuters in Betatown stay, and 50% move to Alphaville.

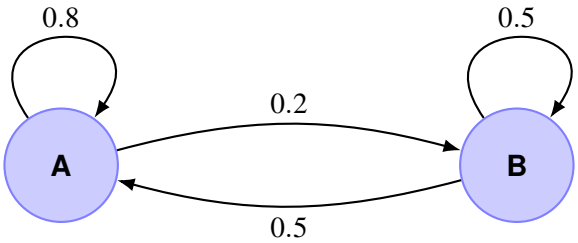


Figure 1: The two-city commuter system. Each edge label represents the fraction of commuters that follow that link at each time step.

Collecting these fractions gives the state transition matrix

$$\mathbf{A} = \begin{bmatrix} 0.8 & 0.5 \\ 0.2 & 0.5 \end{bmatrix}$$

Notice how we built the matrix: the entry $\mathbf{A}_{ij}$ is the fraction of the commuters in city j who move to city i. Column 1 describes where people from Alphaville go (80% stay, 20% leave), and column 2 describes where people from Betatown go (50% go to Alphaville, 50% stay).

If we start with $\mathbf{q}[0] = \begin{bmatrix} 1000 \\ 600 \end{bmatrix}$, then

$$\mathbf{q}[1] = \begin{bmatrix} 0.8 & 0.5 \\ 0.2 & 0.5 \end{bmatrix} \begin{bmatrix} 1000 \\ 600 \end{bmatrix} = \begin{bmatrix} 1100 \\ 500 \end{bmatrix}, \qquad \mathbf{q}[2] = \begin{bmatrix} 0.8 & 0.5 \\ 0.2 & 0.5 \end{bmatrix} \begin{bmatrix} 1100 \\ 500 \end{bmatrix} = \begin{bmatrix} 1130 \\ 470 \end{bmatrix},$$

and we can keep iterating to find how many commuters are in each city on any later day.

10.3.1 Interlude: Graphs and PageRank

Let's take a closer look at the diagram in Figure 1. It is an example of a **directed graph**: a collection of **nodes** (here, the two cities) joined by **directed edges** (the arrows), each pointing from one node to another. Labeling every edge with the fraction of the population that follows it lets us read the state transition matrix directly off the graph, since $\mathbf{A}_{ij}$ is the weight on the edge from node j to node i, or 0 if there is no such edge. The same recipe turns any network into a dynamical system.

One famous example is the internet. We treat each web page as a node and draw an edge from page j to page i whenever page j links to page i. Now imagine a large population of web surfers who, at every time step, each click a random link on their current page. Each page then splits its visitors equally among the pages it links to, so if page j has d_j outgoing links, every edge leaving page j carries weight $1/d_j$. We call the resulting matrix the **connectivity matrix** of the network. In the three-page network of Figure 2, page 1 links to pages 2 and 3, page 2 links to page 3, and page 3 links back to page 1, so the fraction of surfers on each page evolves as $\mathbf{q}[n+1] = \mathbf{A}\mathbf{q}[n]$ with

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & 1 \\ \frac{1}{2} & 0 & 0 \\ \frac{1}{2} & 1 & 0 \end{bmatrix}.$$

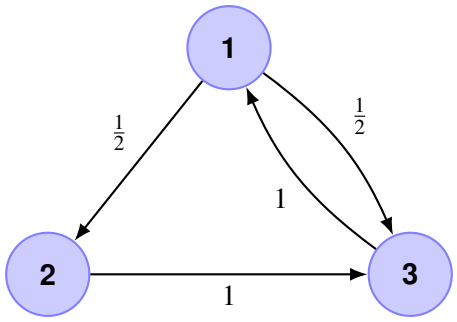


Figure 2: A network of three web pages. An edge from page j to page i means page j links to page i, and its weight is the fraction of page j's visitors who follow that link.

A page that many surfers end up on in the long run is, intuitively, an important page. This is the idea behind Google's **PageRank** algorithm, which ranks web pages by the distribution of surfers that no longer changes from one time step to the next. We will see how to find this distribution, called the *equilibrium state*, in Section 10.6.

10.4 Column-Stochastic Matrices and Conservation

The state transition matrices in the commuter and web examples have something in common: all of their entries are non-negative, and each of their columns sums to 1. Why is that? Column j lists the fractions of whatever is at node j that move to each node, and since nothing is created or lost along the way, these fractions must add up to 1. We see matrices like this whenever the state represents a distribution (of population, probability, etc.) that is redistributed at each time step.

Definition 10.2 (Column-Stochastic Matrix): A matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$ is **column-stochastic** if:

1. All entries are non-negative: $\mathbf{A}_{ij} \geq 0$ for all i, j.
2. Each column sums to one:
$$\sum_{i=1}^{N} \mathbf{A}_{ij} = 1 \quad \text{for all } j.$$

The term "stochastic" comes from probability theory. We can also read column 1 of the commuter matrix as saying that a commuter currently in Alphaville has an 80% chance of staying and a 20% chance of moving to Betatown.

We can write the column-sum condition compactly using the all-ones row vector $\mathbf{1}^\top = \begin{bmatrix} 1 & 1 & \cdots & 1 \end{bmatrix}$:

$$\mathbf{1}^\top \mathbf{A} = \begin{bmatrix} 1 & \cdots & 1 \end{bmatrix} \begin{bmatrix} | & & | \\ \mathbf{a}_1 & \cdots & \mathbf{a}_N \\ | & & | \end{bmatrix} = \begin{bmatrix} \mathbf{1}^\top \mathbf{a}_1 & \cdots & \mathbf{1}^\top \mathbf{a}_N \end{bmatrix}$$

where $\mathbf{a}_j$ denotes the j-th column of $\mathbf{A}$. If every column sums to one, then each entry of the resulting row vector is 1, giving:

$$\mathbf{1}^\top \mathbf{A} = \mathbf{1}^\top \tag{4}$$

Column-stochasticity also guarantees something less obvious: $\mathbf{A}$ has an **eigenvalue of 1**. To see why, let's transpose (4) to get $\mathbf{A}^\top \mathbf{1} = \mathbf{1}$, which says that $\mathbf{1}$ is an eigenvector of $\mathbf{A}^\top$ with eigenvalue 1. Recall that a matrix and its transpose have the same eigenvalues, so $\mathbf{A}$ has eigenvalue 1 as well.

10.4.1 Conservation Property

An important consequence of column-stochasticity is that the "total" of the system is conserved. If we define the total as $T[n] = \mathbf{1}^\top \mathbf{q}[n] = \sum_{k=1}^{N} q_k[n]$, then

$$T[n+1] = \mathbf{1}^\top \mathbf{q}[n+1] = \mathbf{1}^\top \mathbf{A}\mathbf{q}[n] = (\mathbf{1}^\top \mathbf{A})\mathbf{q}[n] = \mathbf{1}^\top \mathbf{q}[n] = T[n] \tag{5}$$

So the total stays the same at every time step. In the commuter example, the total number of commuters $q_1[n] + q_2[n] = 1600$ never changes: commuters move between cities, but none appear or disappear.

What if something is added to the system from outside? If there is an external input $\mathbf{x}[n]$, so that $\mathbf{q}[n+1] = \mathbf{A}\mathbf{q}[n] + \mathbf{x}[n]$, the same calculation shows that the conservation law becomes

$$T[n+1] = T[n] + \mathbf{1}^\top \mathbf{x}[n] \tag{6}$$

so the total changes only by the amount the input adds (or removes). We will study systems with external inputs in more detail in Section 10.7.

10.5 Using Diagonalization to Analyze State Evolution

In the scalar case, the solution $m[n] = a^n m[0]$ is easy to compute for any n. In the vector case, we need $\mathbf{A}^n$, and computing high powers of a matrix directly is expensive. Fortunately, when $\mathbf{A}$ is diagonalizable, there is a much better way.

Recall that $\mathbf{A} \in \mathbb{R}^{N \times N}$ is *diagonalizable* if it has N linearly independent eigenvectors $\mathbf{v}_1, \ldots, \mathbf{v}_N$, with eigenvalues $\lambda_1, \ldots, \lambda_N$. If we let $\mathbf{V} = \begin{bmatrix} \mathbf{v}_1 & \cdots & \mathbf{v}_N \end{bmatrix}$ and let $\mathbf{\Lambda}$ be the diagonal matrix of corresponding eigenvalues, then $\mathbf{V}$ is invertible and $\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}$. Let's use this to compute powers of $\mathbf{A}$:

$$\mathbf{A}^2 = (\mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1})(\mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}) = \mathbf{V}\mathbf{\Lambda}(\mathbf{V}^{-1}\mathbf{V})\mathbf{\Lambda}\mathbf{V}^{-1} = \mathbf{V}\mathbf{\Lambda}^2\mathbf{V}^{-1}$$
$$\mathbf{A}^3 = \mathbf{A} \cdot \mathbf{A}^2 = (\mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1})(\mathbf{V}\mathbf{\Lambda}^2\mathbf{V}^{-1}) = \mathbf{V}\mathbf{\Lambda}^3\mathbf{V}^{-1}$$

The $\mathbf{V}^{-1}\mathbf{V}$ in the middle cancels at every step, so extending the pattern gives

$$\mathbf{A}^n = \mathbf{V}\mathbf{\Lambda}^n\mathbf{V}^{-1}, \quad \text{where} \quad \mathbf{\Lambda}^n = \begin{bmatrix} \lambda_1^n & & 0 \\ & \ddots & \\ 0 & & \lambda_N^n \end{bmatrix} \tag{7}$$

Since $\mathbf{\Lambda}$ is diagonal, raising it to the n-th power simply raises each diagonal entry to the n-th power.

10.5.1 Writing the State in Terms of Eigenvectors

The formula $\mathbf{A}^n = \mathbf{V}\mathbf{\Lambda}^n\mathbf{V}^{-1}$ is most useful when we apply it to the initial state and think about what each factor does. Using the eigendecomposition, we can write the state in a form that makes each factor's role clear. Since $\mathbf{A}$ has N linearly independent eigenvectors, they form a basis for $\mathbb{R}^N$, so we can write any initial state as a linear combination of them:

$$\mathbf{q}[0] = \alpha_1\mathbf{v}_1 + \alpha_2\mathbf{v}_2 + \cdots + \alpha_N\mathbf{v}_N = \mathbf{V}\boldsymbol{\alpha}, \tag{8}$$

where the coefficients are $\boldsymbol{\alpha} = \mathbf{V}^{-1}\mathbf{q}[0]$. If we name $\mathbf{W} = \mathbf{V}^{-1}$ and write it in terms of its rows $\mathbf{w}_1^\top, \ldots, \mathbf{w}_N^\top$, then each coefficient is $\alpha_k = \mathbf{w}_k^\top\mathbf{q}[0]$.

Why is this decomposition useful? Because $\mathbf{A}$ acts on each eigenvector by simply scaling it. Since $\mathbf{A}\mathbf{v}_k = \lambda_k\mathbf{v}_k$, applying $\mathbf{A}$ a total of n times gives $\mathbf{A}^n\mathbf{v}_k = \lambda_k^n\mathbf{v}_k$, and by linearity we can evolve each term of (8) separately and add the results:

$$\mathbf{q}[n] = \mathbf{A}^n\mathbf{q}[0] = \sum_{k=1}^{N} \alpha_k\,\mathbf{A}^n\mathbf{v}_k = \sum_{k=1}^{N} \alpha_k\,\lambda_k^n\,\mathbf{v}_k. \tag{9}$$

This is exactly what $\mathbf{q}[n] = \mathbf{V}\mathbf{\Lambda}^n\mathbf{V}^{-1}\mathbf{q}[0]$ computes when read from right to left: $\mathbf{V}^{-1}$ finds the coefficients α_k, $\mathbf{\Lambda}^n$ scales the k-th coefficient by λ_k^n, and $\mathbf{V}$ recombines the scaled eigenvectors into the state.

In other words, the state at any time is a sum of N terms, one for each eigenvalue. These terms evolve independently of one another: the coefficient on $\mathbf{v}_k$ is multiplied by λ_k at every step, exactly like the scalar system $m[n+1] = a\,m[n]$ with $a = \lambda_k$. Diagonalization therefore splits the coupled N-dimensional problem into N independent scalar problems, and each piece of the term $\alpha_k\lambda_k^n\mathbf{v}_k$ with eigenvalue λ_k tells us something different about the system:

- The **eigenvector** $\mathbf{v}_k$ sets the *spatial pattern* of the term: how it is distributed across the state variables.
- The **eigenvalue** λ_k sets the *temporal behavior*: whether the term grows, decays, or oscillates.
- The **coefficient** α_k sets *how much* of $\mathbf{v}_k$ is present in the initial state.

Let's see the decomposition in action. Consider the system with

$$\mathbf{A} = \begin{bmatrix} 1 & \frac{1}{2} \\ \frac{1}{4} & \frac{3}{4} \end{bmatrix},$$

whose eigenpairs are $\lambda_1 = \frac{5}{4}$, $\mathbf{v}_1 = \begin{bmatrix} 2 & 1 \end{bmatrix}^\top$ and $\lambda_2 = \frac{1}{2}$, $\mathbf{v}_2 = \begin{bmatrix} -1 & 1 \end{bmatrix}^\top$. The initial state $\mathbf{q}[0] = \begin{bmatrix} 1 & 2 \end{bmatrix}^\top$ decomposes as $\mathbf{v}_1 + \mathbf{v}_2$, so $\alpha_1 = \alpha_2 = 1$ and

$$\mathbf{q}[n] = \left(\tfrac{5}{4}\right)^n \mathbf{v}_1 + \left(\tfrac{1}{2}\right)^n \mathbf{v}_2.$$

Figure 3 shows the first few steps. At each step, the term with eigenvalue $\frac{5}{4}$ grows by a factor of $\frac{5}{4}$ while the term with eigenvalue $\frac{1}{2}$ shrinks by half, and the new state is recomposed from the two scaled eigenvectors. As n increases, the $\mathbf{v}_2$ term dies out and the state turns to point along $\mathbf{v}_1$, after which it simply grows by $\frac{5}{4}$ at every step.

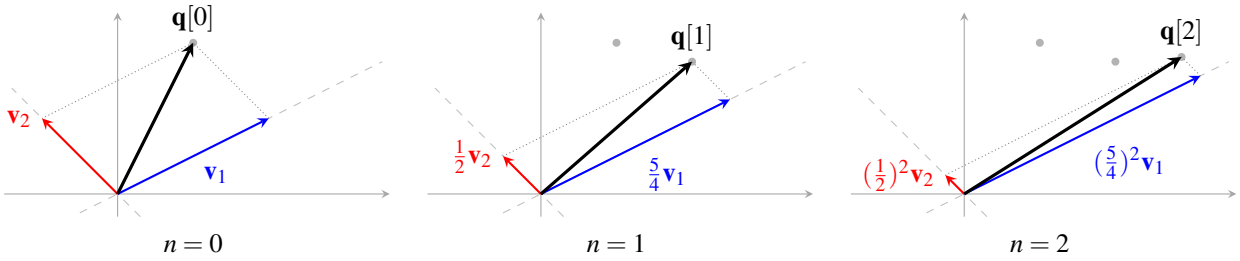


Figure 3: The state $\mathbf{q}[n]$ written in terms of the eigenvectors of $\mathbf{A}$, for the first three steps. The dashed gray lines are the eigenvector directions. At each step, $\mathbf{A}$ scales the $\mathbf{v}_1$ component (blue) by $\frac{5}{4}$ and the $\mathbf{v}_2$ component (red) by $\frac{1}{2}$, and the state (black) is their sum. The gray dots trace the earlier states.

10.6 Long-Term Behavior and Stability

Now that we can compute $\mathbf{A}^n$ efficiently, we can ask what happens to the state as $n \to \infty$. Writing the state in terms of eigenvectors as in (9),

$$\mathbf{q}[n] = \sum_{k=1}^{N} \alpha_k \lambda_k^n \mathbf{v}_k,$$

we see that each term behaves like a scalar system. Recall from Section 10.1.1 that the solution $m[n] = a^n m[0]$ of a scalar system decays to zero when $|a| < 1$, keeps a constant magnitude when $|a| = 1$, and grows without bound when $|a| > 1$. The same is true here, so the long-term behavior of the term with eigenvalue λ_k depends only on $|\lambda_k|$:

- $|\lambda_k| < 1$: The term **decays** to zero as $n \to \infty$, since $\lambda_k^n \to 0$.
- $|\lambda_k| = 1$: The term **persists**. If $\lambda_k = 1$, the term is constant. If $\lambda_k = -1$ or is complex on the unit circle, the term oscillates without growing or shrinking in magnitude.
- $|\lambda_k| > 1$: The term **grows** without bound, since $|\lambda_k^n| \to \infty$.

We can summarize these cases on the complex plane, as in Figure 4, where the unit circle separates the eigenvalues whose terms decay from those whose terms grow. Eigenvalues off the real axis also make their terms oscillate, but it is the distance $|\lambda_k|$ from the origin that decides whether the oscillation dies out, persists, or blows up.

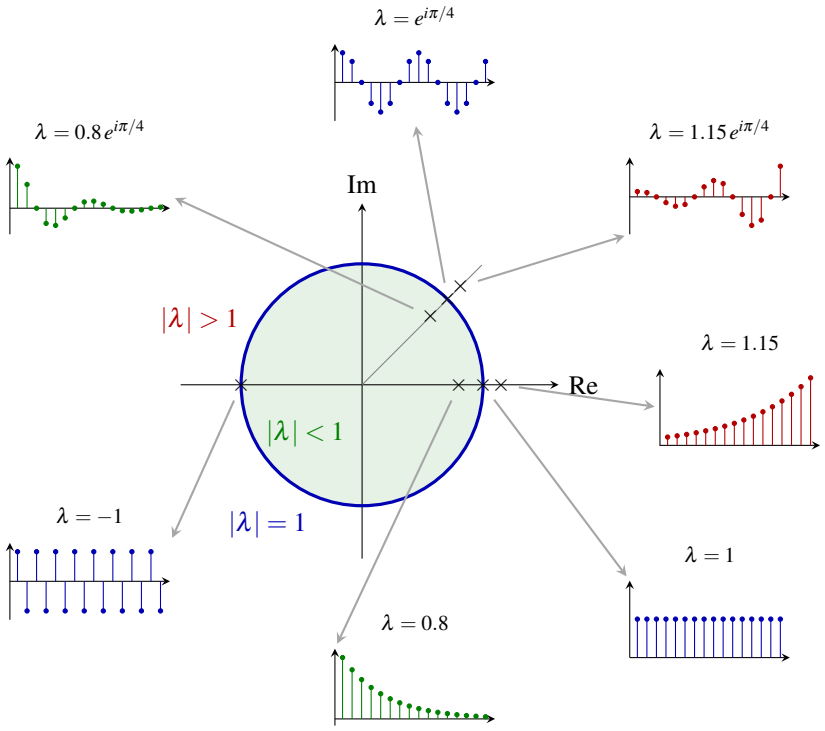


Figure 4: Each panel shows $\mathrm{Re}\{\lambda^n\}$ for $n = 0, \ldots, 15$, for the eigenvalue marked $\times$ on the complex plane. Terms whose eigenvalues lie inside the unit circle decay to zero, terms with eigenvalues on the unit circle persist with constant amplitude, and terms with eigenvalues outside the unit circle grow without bound.

Definition 10.3 (Internal Stability (Asymptotic Stability)): A system is **internally stable** (or **asymptotically stable**) if, without any external input,

$$\lim_{n\to\infty} \mathbf{q}[n] = \mathbf{0}$$

regardless of the initial state $\mathbf{q}[0]$. This holds if and only if every eigenvalue of $\mathbf{A}$ satisfies $|\lambda_k| < 1$, i.e., all eigenvalues lie **strictly inside the unit circle**.

If even a single eigenvalue has $|\lambda_k| > 1$, the term with that eigenvalue will grow without bound, unless the initial state happens to have $\alpha_k = 0$.

Intuitively, an internally stable system eventually forgets where it started: left to itself, the state decays to $\mathbf{0}$ no matter what the initial state was.

Definition 10.4 (Equilibrium State): A vector $\mathbf{q}^*$ is an **equilibrium state** of the system $\mathbf{q}[n+1] = \mathbf{A}\mathbf{q}[n]$ if

$$\mathbf{A}\mathbf{q}^* = \mathbf{q}^*$$

In other words, once the system reaches $\mathbf{q}^*$, it stays there forever. An equilibrium state is an eigenvector of $\mathbf{A}$ corresponding to eigenvalue $\lambda = 1$. As discussed earlier, every column-stochastic matrix has $\lambda = 1$ as an eigenvalue, so it always has an equilibrium.

Example 10.2 (Three Connected Reservoirs): Three reservoirs are connected in a row by channels, and at each time step water flows between them as shown in Figure 5. Let $q_k[n]$ be the volume of water (in megaliters) in reservoir k at time n. Reading the matrix off the diagram as we did for the commuter system, the weights on the edges leaving reservoir j fill column j, so $\mathbf{q}[n+1] = \mathbf{A}\mathbf{q}[n]$ with

$$\mathbf{A} = \begin{bmatrix} 3/4 & 1/4 & 0 \\ 1/4 & 1/2 & 1/4 \\ 0 & 1/4 & 3/4 \end{bmatrix}.$$

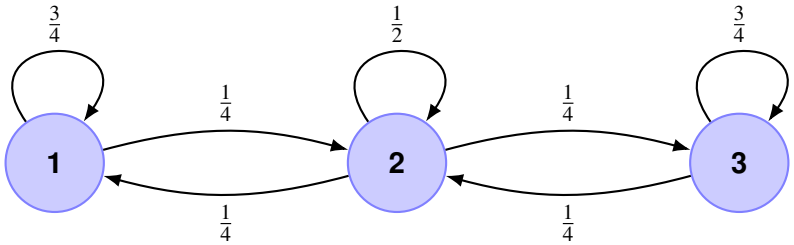


Figure 5: The three-reservoir system. Each edge label is the fraction of the water in a reservoir that follows that channel at each time step.

We can check that its eigenpairs are

$$\lambda_1 = 1,\ \mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}; \qquad \lambda_2 = \tfrac{3}{4},\ \mathbf{v}_2 = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}; \qquad \lambda_3 = \tfrac{1}{4},\ \mathbf{v}_3 = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}.$$

Inverting $\mathbf{V} = \begin{bmatrix} \mathbf{v}_1 & \mathbf{v}_2 & \mathbf{v}_3 \end{bmatrix}$ gives the rows

$$\mathbf{w}_1^\top = \tfrac{1}{3}\begin{bmatrix} 1 & 1 & 1 \end{bmatrix}, \qquad \mathbf{w}_2^\top = \tfrac{1}{2}\begin{bmatrix} 1 & 0 & -1 \end{bmatrix}, \qquad \mathbf{w}_3^\top = \tfrac{1}{6}\begin{bmatrix} 1 & -2 & 1 \end{bmatrix}.$$

Suppose all 60 megaliters start in the first reservoir, so that $\mathbf{q}[0] = \begin{bmatrix} 60 & 0 & 0 \end{bmatrix}^\top$. Then the coefficients are

$$\alpha_1 = \mathbf{w}_1^\top \mathbf{q}[0] = 20, \qquad \alpha_2 = \mathbf{w}_2^\top \mathbf{q}[0] = 30, \qquad \alpha_3 = \mathbf{w}_3^\top \mathbf{q}[0] = 10,$$

so by (9),

$$\mathbf{q}[n] = 20\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} + 30\left(\tfrac{3}{4}\right)^n \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} + 10\left(\tfrac{1}{4}\right)^n \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}.$$

As $n \to \infty$, the two terms with $|\lambda_k| < 1$ decay to zero and only the term with eigenvalue $\lambda_1 = 1$ survives:

$$\mathbf{q}[n] \to \begin{bmatrix} 20 \\ 20 \\ 20 \end{bmatrix}.$$

This limit is an equilibrium state, since it is an eigenvector of $\mathbf{A}$ with eigenvalue 1. The water levels equalize, with each reservoir settling at 20 megaliters in steady-state.

Is this system internally stable? No! The eigenvalue $\lambda_1 = 1$ is not strictly inside the unit circle, so the state does not go to $\mathbf{0}$. Instead, it converges to a nonzero equilibrium determined by the initial condition.

In addition, since $\mathbf{A}$ is column-stochastic, the total volume $\mathbf{1}^\top \mathbf{q}[n] = 60$ never changes. The decaying terms satisfy $\mathbf{1}^\top \mathbf{v}_2 = \mathbf{1}^\top \mathbf{v}_3 = 0$, so they carry no net water, and the entire total sits in the term with eigenvalue $\lambda_1 = 1$.

10.7 General State Space Models

So far, our systems have evolved autonomously: the state at the next time step depends only on the current state. In practice, systems often have external inputs (forces, signals, controls) that affect their evolution, and we observe the system through some **output**.

Adding inputs and outputs to our model gives the general discrete-time state space model:

$$\mathbf{q}[n+1] = \mathbf{A}\mathbf{q}[n] + \mathbf{B}\mathbf{x}[n] \tag{10}$$

$$\mathbf{y}[n] = \mathbf{C}\mathbf{q}[n] + \mathbf{D}\mathbf{x}[n] \tag{11}$$

with initial state $\mathbf{q}[0]$. The signals are:

- $\mathbf{q}[n] \in \mathbb{R}^N$: the **state vector**,
- $\mathbf{x}[n] \in \mathbb{R}^M$: the **input vector**,
- $\mathbf{y}[n] \in \mathbb{R}^L$: the **output vector**,

and the system is described by four matrices:

Matrix	Name	Size	Maps
$\mathbf{A}$	state transition matrix	$N \times N$	state $\to$ next state
$\mathbf{B}$	input matrix	$N \times M$	input $\to$ next state
$\mathbf{C}$	output matrix	$L \times N$	state $\to$ output
$\mathbf{D}$	feedthrough matrix	$L \times M$	input $\to$ output

We call (10) the **state evolution equation**, since it describes the dynamics of the system, and (11) the **output equation**. Both depend only on the current state $\mathbf{q}[n]$ and current input $\mathbf{x}[n]$. In this course, $\mathbf{C}$ and $\mathbf{D}$ come up less often than $\mathbf{A}$ and $\mathbf{B}$: we can usually observe the state directly, so $\mathbf{C} = \mathbf{I}$ and $\mathbf{D} = \mathbf{0}$, and the output is simply $\mathbf{y}[n] = \mathbf{q}[n]$. The general output equation matters when we can only measure some combination of the state variables, or when the input affects the output directly.

10.8 Solving the General State-Space Equations

Now that we have the general model, how do we solve it? Just as we did without inputs in Section 10.3, let's iterate the state equation forward from the initial state:

$$\mathbf{q}[1] = \mathbf{A}\mathbf{q}[0] + \mathbf{B}\mathbf{x}[0]$$

$$\mathbf{q}[2] = \mathbf{A}\mathbf{q}[1] + \mathbf{B}\mathbf{x}[1] = \mathbf{A}^2\mathbf{q}[0] + \mathbf{A}\mathbf{B}\mathbf{x}[0] + \mathbf{B}\mathbf{x}[1]$$

$$\mathbf{q}[3] = \mathbf{A}\mathbf{q}[2] + \mathbf{B}\mathbf{x}[2] = \mathbf{A}^3\mathbf{q}[0] + \mathbf{A}^2\mathbf{B}\mathbf{x}[0] + \mathbf{A}\mathbf{B}\mathbf{x}[1] + \mathbf{B}\mathbf{x}[2]$$

Each input $\mathbf{x}[k]$ enters the state through $\mathbf{B}$ and is then multiplied by $\mathbf{A}$ once for every step that follows, so the pattern gives the general state solution:

$$\boxed{\mathbf{q}[n] = \mathbf{A}^n\mathbf{q}[0] + \sum_{k=0}^{n-1} \mathbf{A}^{n-1-k}\mathbf{B}\mathbf{x}[k]} \tag{12}$$

Substituting this into the output equation (11) gives the output:

$$\mathbf{y}[n] = \underbrace{\mathbf{C}\mathbf{A}^n\mathbf{q}[0]}_{\text{ZIR}} + \underbrace{\mathbf{C}\sum_{k=0}^{n-1} \mathbf{A}^{n-1-k}\mathbf{B}\mathbf{x}[k] + \mathbf{D}\mathbf{x}[n]}_{\text{ZSR}}. \tag{13}$$

Both the state and the output split into two parts:

- The **zero-input response (ZIR)** is the response to the initial condition alone, with the input turned off ($\mathbf{x}[n] = \mathbf{0}$ for all n). Recall from Section 10.3 that without inputs the state is $\mathbf{A}^n\mathbf{q}[0]$, which is exactly the first term of (12).
- The **zero-state response (ZSR)** is the response to the input alone, with the system starting with initial conditions of 0 ($\mathbf{q}[0] = \mathbf{0}$). In the state, each term $\mathbf{A}^{n-1-k}\mathbf{B}\mathbf{x}[k]$ is the contribution of the input at time k, propagated forward $n-1-k$ steps by $\mathbf{A}$.

In other words, the response of the system is the sum of what the initial state would do on its own and what the input would do on its own.